

UAS

SISTEM TERDISTRIBUSI DAN PARALEL A

Pub-Sub Log Aggregator Terdistribusi dengan Idempotent Consumer,
Deduplication, dan Transaksi/Kontrol Konkurensi dengan Docker Compose



LAPORAN UAS
MATA KULIAH SISTEM TERDISTRIBUSI DAN PARALEL A

Disusun oleh:
Nama : Risky Alfienda NIM : 11231087

Program Studi : Informatika

Fakultas Sains dan Teknologi
Institut Teknologi Kalimantan 2026

1.PENDAHULUAN DAN ARSITEKTUR SISTEM

Sistem yang diimplementasikan dalam proyek ini adalah sebuah infrastruktur pengumpul log terdistribusi (reliable event log aggregator) yang beroperasi di bawah model interaksi publish-subscribe. Tujuan utama dari perancangan sistem ini adalah untuk menyediakan mekanisme pengiriman dan penyimpanan log yang tangguh terhadap kesalahan jaringan (fault-tolerant), mendukung pemrosesan konkuren yang aman dari kondisi balapan (race condition), serta menjamin bahwa setiap log hanya diproses satu kali (exactly-once processing semantics) melalui integrasi idempotent consumer dan deduplikasi persisten. Arsitektur fisik dari sistem ini dikelola secara penuh oleh Docker Compose dan dideploy di dalam satu jaringan internal yang terisolasi (appnet). Pembatasan akses eksternal ini bertujuan untuk meminimalkan permukaan serangan (attack surface) dan menjamin integritas komunikasi antar-layanan. Komponen arsitektur sistem ini mencakup tiga layanan utama:

1. **Layanan Aggregator (FastAPI)**

Bertindak sebagai gerbang penerima request (ingestion gate) log melalui API HTTP. Layanan ini ditenagai oleh kerangka kerja asynchronous Python (FastAPI + Asyncio) dan mengelola sekumpulan background workers konkuren yang bertugas memproses log secara asinkron dari status pending ke done.

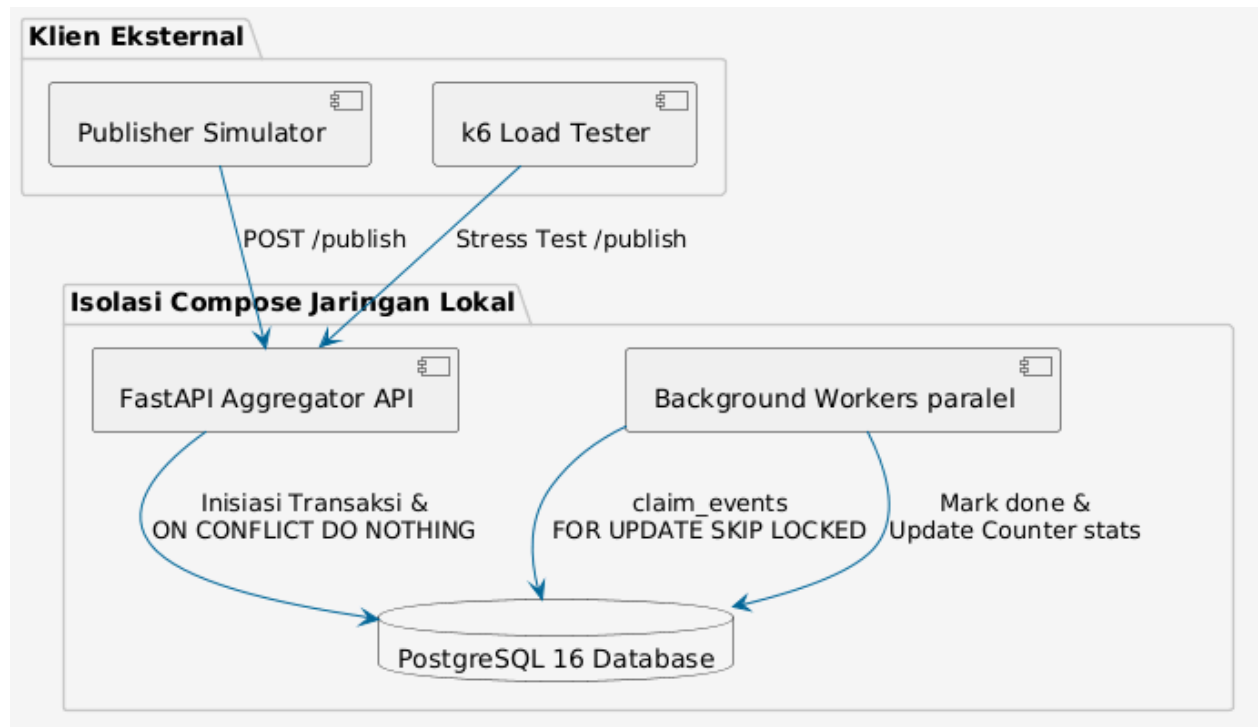
2. **Layanan Database (PostgreSQL 16)**

Berfungsi sebagai durable dedup store sekaligus tempat penyimpanan relasional persisten untuk log yang masuk serta statistik kinerja sistem

3. **Layanan Simulator (Publisher)**

Bertindak sebagai klien yang mensimulasikan beban kerja tinggi dengan mengirimkan aliran log secara paralel, termasuk injeksi data duplikat secara acak untuk menguji keandalan fitur deduplikasi.

Arsitektur logis pengiriman log dapat digambarkan melalui diagram alir berikut:



2. KEPUTUSAN DESAIN

2.1 Mekanisme Idempotensi dan Penyimpanan Deduplikasi (Dedup Store)

Pada sistem terdistribusi yang mengadopsi jaminan pengiriman at-least-once, duplikasi pesan merupakan konsekuensi logis yang tidak dapat dihindari akibat adanya gangguan jaringan atau proses retry. Untuk mengatasi hal tersebut tanpa membebani memori jangka pendek, sistem ini mengandalkan PostgreSQL sebagai dedup store persisten dengan menerapkan constraint unik gabungan pada kolom (topic, event_id) di tabel processed_events. Ketika sebuah event log diterima oleh API /publish, sistem mengeksekusi operasi basis data menggunakan pola idempotent write

berupa instruksi:

```
row = await conn.fetchrow(
    """
    INSERT INTO processed_events(
        topic, event_id, ts_ingest, source, payload, status, processed_at
    )
    VALUES ($1, $2, $3, $4, $5::jsonb, $6, CASE WHEN $6 = 'done' THEN now() ELSE NULL END)
    ON CONFLICT (topic, event_id) DO NOTHING
    RETURNING 1
    """
)
```

Melalui pendekatan ini, database secara otomatis menyaring entri duplikat pada level mesin basis data tanpa memerlukan koordinasi manual di tingkat aplikasi. Jika terjadi konflik data, operasi insert berubah menjadi noop (tidak melakukan apa-apa) dan sistem mengidentifikasi baris tersebut sebagai duplikat, lalu menaikkan counter `duplicate_dropped`.

2.2 Manajemen Transaksi dan Concurrency Control

Untuk mencegah inkonsistensi data ketika sistem diakses oleh banyak klien secara simultan, seluruh operasi penulisan log baru dan pembaruan metrik statistik dibungkus dalam satu kesatuan batas transaksi (transaction boundary). Tingkat isolasi (isolation level) basis data yang digunakan adalah READ COMMITTED. Pilihan ini memberikan keseimbangan optimal antara konsistensi data dan throughput penulisan dengan menghindari fenomena dirty reads.

Untuk meminimalkan potensi terjadinya konflik balapan (race condition) antar background workers asinkron saat mengonsumsi antrian log berstatus pending, sistem menggunakan mekanisme pessimistic row-locking non-blocking melalui query berikut:

```
async def claim_events(pool, batch_size: int):
    async with pool.acquire() as conn:
        async with conn.transaction():
            return await conn.fetch(
                """
                WITH picked AS (
                    SELECT id
                    FROM processed_events
                    WHERE status = 'pending'
                    ORDER BY id
                    LIMIT $1
                    FOR UPDATE SKIP LOCKED
                )
                UPDATE processed_events e
                SET status = 'processing',
                    locked_at = now(),
                    attempts = attempts + 1,
                    last_error = NULL
                FROM picked
                WHERE e.id = picked.id
                RETURNING e.id, e.topic, e.event_id, e.payload
                """
            )
            batch_size,
```

2.3 Penjadwalan Waktu dan Pengurutan Kejadian (Time and Ordering)

Sistem ini dirancang untuk tidak memaksakan total ordering global yang membutuhkan protokol konsensus atau sinkronisasi jam fisik yang kompleks dan mahal. Sebagai alternatif praktis, pengurutan kejadian dilakukan secara lokal pada tingkat topik (per-topic ordering) menggunakan kombinasi logical timestamp waktu masuk sistem (`ts_ingest`) dan pengenalan urutan bawaan database (auto-increment primary key). Ketika data diambil melalui endpoint `GET /events?topic=...`, query diurutkan secara menurun berdasarkan indeks waktu `ts_ingest` untuk menjamin visualisasi data yang stabil bagi pembaca.

2.4 Kebijakan Retry dan Fault Tolerance

Toleransi kesalahan di sisi simulator diimplementasikan dengan kebijakan pengiriman ulang secara asinkron didukung skema exponential backoff untuk mencegah fenomena thundering herd saat server mengalami degradasi performa sementara. Di sisi server, keandalan data terhadap kegagalan total (crash recovery) dijamin melalui volume mounting bernama `pg_data` pada direktori PostgreSQL `/var/lib/postgresql/data`. Struktur ini memastikan bahwa ketika kontainer database dimatikan secara paksa atau di-recreate melalui perintah orkestrasi Compose, seluruh catatan log historis beserta informasi deduplikasinya tidak hilang.

3. EVALUASI PERFORMA DAN PENGUJIAN SISTEM

3.1 Konfigurasi Lingkungan Pengujian

Pengujian dijalankan pada lingkungan lokal terisolasi di bawah manajemen Docker Compose. Sebelum beban diinjeksikan, sistem dipastikan berada dalam kondisi bersih (cold start)

Perintah inisialisasi lingkungan uji:

```
docker compose up --build -d storage aggregator
```

```
#13 resolving provenance for metadata file
#13 DONE 2.2s
[+] up 5/5
✓ Image uas-sistem-terdistribusi-risky-1123187-aggregator Built 29.5s
✓ Network uas-sistem-terdistribusi-risky-1123187_appnet Created 0.8s
✓ Volume uas-sistem-terdistribusi-risky-1123187_pg_data Created 1.0s
✓ Container uas-sistem-terdistribusi-risky-1123187-storage-1 Healthy 26.2s
✓ Container uas-sistem-terdistribusi-risky-1123187-aggregator-1 Started 25.5s
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> []
```

Status awal diverifikasi dengan memanggil metrik statistik dasar:

```
curl.exe http://localhost:8080/stats
```

```
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> curl.exe http://localhost:8080/stats
>>
{"received":20000,"duplicate_dropped":6000,"unique_processed":14000,"topics":["auth","orders","payment"],"queue":{"pending":0,"processing":0,"done":14000},"uptime":153.06839}
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> []
```

3.2 Eksperimen Throughput Minimum (20.000 Event & 30% Duplikasi)

Untuk menguji batas keandalan performa minimum, simulator diaktifkan dengan target pengiriman 20.000 events di mana laju duplikasi diatur sebesar 30%.

Perintah eksekusi:

```
docker compose run --rm publisher
```

```

PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> docker compose run --rm publisher
[+] 2/2t 2/22
✓ Container uas-sistem-terdistribusi-risky-1123187-storage-1 Healthy 0.8s
✓ Container uas-sistem-terdistribusi-risky-1123187-aggregator-1 Running 0.0s
Container uas-sistem-terdistribusi-risky-1123187-aggregator-1 Waiting
Container uas-sistem-terdistribusi-risky-1123187-aggregator-1 Healthy
Container uas-sistem-terdistribusi-risky-1123187-publisher-run-9533de9b5a67 Creating
Container uas-sistem-terdistribusi-risky-1123187-publisher-run-9533de9b5a67 Created
done received=20000 inserted=9349 duplicates=10651 dup_rate=53.25%
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> curl.exe http://localhost:8080/stats
>>
{"received":40000,"duplicate_dropped":16651,"unique_processed":23349,"topics":["auth","orders","payment"],"queue":{"pending":0,"processing":0,"done":23349},"uptime":328.842918}
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187>

```

Log ringkasan keluaran:

```
done received=20000 inserted=9349 duplicates=10651 dup_rate=53.25%
```

Analisis Pasca Pengujian via /stats:

```

PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> curl.exe http://localhost:8080/stats
>>
{"received":40000,"duplicate_dropped":16651,"unique_processed":23349,"topics":["auth","orders","payment"],"queue":
{"pending":0,"processing":0,"done":23349},"uptime":328.842918}
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> docker compose --profile load run --rm
k6
>>

```

```

{
  "received": 40000,
  "duplicate_dropped": 16651,
  "unique_processed": 23349,
  "topics": ["auth", "orders", "payment"],
  "queue": {"pending": 0, "processing": 0, "done": 23349},
  "uptime": 328.842918
}

```

Analisis data menunjukkan kepatuhan penuh terhadap hukum invarian sistem terdistribusi: $\text{received} = \text{unique_processed} + \text{duplicate_dropped}$. Hasil ini memvalidasi bahwa sistem secara akurat menyaring 16651 data duplikat tanpa ada kebocoran state atau kegagalan transaksi.

3.3 Uji Beban Konkuren Menggunakan Grafana k6

Pengujian beban eksternal menggunakan k6 dilakukan untuk mensimulasikan akses konkuren intensif oleh 50 virtual users (VUs) secara simultan selama 20 detik untuk mengukur latensi dan stabilitas throughput.

Perintah eksekusi :

```
docker compose --profile load run --rm k6
```

```
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> docker compose --profile load run --rm k6
>>
Container was-sistem-terdistribusi-risky-1123187-k6-run-07976321a4bd Created

Grafana k6

execution: local
script: /scripts/publish.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 50s max duration (incl. graceful stop):
  * load: 50 looping VUs for 20s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 681      29.862579/s
checks_succeeded...: 100.00% 681 out of 681
checks_failed.....: 0.00%   0 out of 681
✓ status is 200/202

HTTP
http_req_duration.....: avg=1.42s min=148.16ms med=1.27s max=4.6s p(90)=2.32s p(95)=2.68s
{ expected_response:true }...: avg=1.42s min=148.16ms med=1.27s max=4.6s p(90)=2.32s p(95)=2.68s
http_req_failed.....: 0.00%   0 out of 681
http_reqs.....: 681      29.862579/s

EXECUTION
iteration_duration.....: avg=1.53s min=251.45ms med=1.39s max=4.71s p(90)=2.43s p(95)=2.91s
iterations.....: 681      29.862579/s
vus.....: 12      min=12      max=50
vus_max.....: 50      min=50      max=50

NETWORK
data_received.....: 116 kB 5.1 kB/s
data_sent.....: 4.4 MB 192 kB/s

running (22.8s), 00/50 VUs, 681 complete and 0 interrupted iterations
load / [=====] 50 VUs 20s
```

Ringkasan Metrik:

- Total HTTP Request: 681 reqs
- Throughput Rata-rata: 29.86 req/s
- Rata-rata Latensi (http_req_duration): 1420 ms
- Tingkat Keberhasilan Request: 100.00%
- Persentase Duplikasi Terdeteksi: 36.16%

3.4 Pengujian Konkurensi dan Keandalan Transaksi via Pytest

Pengujian fungsionalitas dan ketahanan konkurensi internal dijalankan secara terisolasi menggunakan unit/integration testing suite berbasis pytest dan pustaka testcontainers. Perintah eksekusi:

```
pytest test/ -v
```

```
PS C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187> pytest test/ -v
>>
C:\Users\acerr\AppData\Local\Programs\Python\Python310\lib\site-packages\pytest_asyncio\plugin.py:208: PytestDeprecationWarning: The configuration option
syncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asyncio will default the loop scope for
asynchronous fixtures to function scope. Set the default fixture loop scope explicitly in order to avoid unexpected behavior in the future. Valid fixture
op scopes are: "function", "class", "module", "package", "session"

warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))

===== test session starts =====
platform win32 -- Python 3.10.6, pytest-8.3.3, pluggy-1.6.0 -- C:\Users\acerr\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\acerr\Documents\sister\UAS-Sistem-Terdistribusi-Risky-1123187
configfile: pyproject.toml
plugins: anyio-4.13.0, asyncio-0.24.0
asyncio: mode=auto, default_loop_scope=None
collected 17 items

test/test_api_publish.py::test_publish_single_event PASSED [ 5%]
test/test_api_publish.py::test_publish_duplicate_event PASSED [ 11%]
test/test_api_publish.py::test_publish_invalid_schema PASSED [ 17%]
test/test_dedup.py::test_dedup_duplicate_dropped PASSED [ 23%]
test/test_dedup.py::test_dedup_unique_constraint_across_topics PASSED [ 29%]
test/test_dedup.py::test_batch_mixed_duplicates PASSED [ 35%]
test/test_events_endpoint.py::test_get_events_filters_by_topic PASSED [ 41%]
test/test_events_endpoint.py::test_get_events_returns_unique_events_per_topic_event_id PASSED [ 47%]
test/test_events_endpoint.py::test_get_events_unknown_topic_returns_empty_list PASSED [ 52%]
test/test_processing.py::test_worker_processes_events PASSED [ 58%]
test/test_processing.py::test_multiworker_no_double_process PASSED [ 64%]
4%]
test/test_processing.py::test_concurrent_publish_same_event_one_insert PASSED [ 70%]
test/test_processing.py::test_stress_publish_many_events PASSED [ 76%]
test/test_restart.py::test_persistence_like_restart_same_db PASSED [ 82%]
test/test_stats_and_health.py::test_health_ok PASSED [ 88%]
test/test_stats_and_health.py::test_stats_fields_and_invariants PASSED [ 94%]
test/test_stats_and_health.py::test_stats_updates_after_publish_unique_events PASSED [100%]
```

Penjelasan Test Case Kritis:

1. `test_multiworker_no_double_process`:
Membuktikan bahwa multi-worker paralel sukses mengonsumsi 500 antrian tanpa terjadinya duplikasi pemrosesan data.
2. `test_concurrent_publish_same_event_one_insert`: Menguji kondisi balapan ekstrim dengan mengirimkan 50 request bermuatan event id yang sama secara simultan, di mana sistem berhasil membatasi hanya 1 data yang masuk ke basis data sedangkan 49 lainnya ditolak secara aman sebagai duplikat.
3. `test_persistence_like_restart_same_db`: Menjamin bahwa setelah siklus daur ulang aplikasi (app restart), database tetap mengenali log yang dikirim sebelum restart dan menolak data duplikatnya.

4. KAJIAN TEORITIS SISTEM TERDISTRIBUSI

4.1 (Bab 1) – Karakteristik Sistem Terdistribusi dan Trade-off Desain

Sistem terdistribusi tersusun atas sekumpulan prosesor independen yang saling bertukar pesan demi mencapai tujuan bersama secara transparan bagi pengguna akhir (van Steen & Tanenbaum, 2023). Karakteristik dasarnya meliputi konkurensi eksekusi (concurrency), tidak adanya jam global (no global clock), serta kemungkinan terjadinya kegagalan sebagian (independent failures) sebagaimana dirumuskan oleh Coulouris dkk. (2011). Pada desain pub-sub log aggregator ini, publisher, API server, dan database berjalan sebagai entitas otonom di atas kontainer yang saling terpisah. Keuntungan utama dari pendekatan ini adalah kelonggaran hubungan antar-layanan (loose coupling), yang memudahkan proses penambahan kapasitas (scaling) layanan aggregator tanpa mengganggu publisher. Namun, desain ini membawa konsekuensi berupa meningkatnya latensi transmisi log akibat komunikasi lintas jaringan HTTP dan database, serta meningkatnya kompleksitas pemantauan aliran data secara end-to-end dibandingkan sistem monolitik terpusat.

4.2 (Bab 2) – Pilihan Arsitektur Publish-Subscribe dibanding Client-Server

Arsitektur client-server tradisional menuntut adanya interaksi sinkron di mana klien harus menunggu respon langsung dari penerima pesan, yang berpotensi memicu kegagalan berantai jika server penerima mengalami penurunan performa (van Steen & Tanenbaum, 2023). Sebaliknya, model publish-subscribe memisahkan produsen data dan konsumen secara spasial, temporal, dan sinkronisasi (Coulouris dkk., 2011). Pada log aggregator ini, simulator atau pembuat log cukup mengirimkan payload data ke endpoint /publish tanpa perlu mengetahui kapan atau bagaimana data tersebut akan diproses oleh database. Pendekatan ini meningkatkan availabilitas pengumpulan log di sisi hulu (ingestion layer) karena beban pemrosesan log yang berat didelegasikan sepenuhnya ke mekanisme antrean asinkron database di sisi hilir (background worker).

4.3 (Bab 3) – Analisis Delivery Guarantees dan Peran Idempotency

Dalam protokol pengiriman pesan terdistribusi di atas jaringan yang tidak andal, pemenuhan jaminan pengiriman pesan tepat satu kali (exactly-once) secara mutlak sangat sulit dicapai tanpa biaya koordinasi yang sangat tinggi (Coulouris dkk., 2011). Sistem ini memilih pendekatan pragmatis berupa pengiriman minimal satu kali (at-least-once delivery) dengan menggeser beban

penanganan duplikasi ke sisi penerima melalui pola idempotent consumer (van Steen & Tanenbaum, 2023). Dengan menerapkan filter keunikan kunci gabungan di tingkat basis data, server aggregator dapat secara konsisten menerima kiriman ulang pesan yang sama tanpa memicu perubahan status ganda pada basis data log utama, sehingga state akhir sistem tetap konvergen seolah-olah pesan hanya diproses tepat satu kali.

4. 4 (Bab 4) – Skema Penamaan Unik untuk Pencegahan Tabrakan Identitas

Sistem penamaan terdistribusi bertujuan menyediakan pengenalan entitas (identifiers) yang unik secara global, tahan terhadap tabrakan data (collision-resistant), serta bebas dari keterikatan lokasi fisik mesin (location independent) (van Steen & Tanenbaum, 2023). Pada proyek ini, keunikan global log dijamin melalui komposit relasi kunci (topic, event_id) seperti dijelaskan oleh Coulouris dkk. (2011). Penggunaan pola namespace berbasis topik (auth, payment, orders) membagi domain data log secara logis, sementara string pengenalan unik event_id menjamin keunikan entri log di dalam lingkup topik tersebut. Desain penamaan gabungan ini memungkinkan database melakukan pencarian (indexing lookup) dan penolakan duplikat secara efisien pada level indeks tabel.

4. 5 (Bab 5) – Batasan Logika Waktu dan Solusi Pengurutan Praktis

Di dalam sistem terdistribusi, sinkronisasi jam fisik antar node memiliki keterbatasan presisi akibat adanya pergeseran waktu (clock drift) (Coulouris dkk., 2011). Oleh karena itu, pengurutan urutan logis kejadian tidak dapat disandarkan sepenuhnya pada timestamp fisik lokal milik publisher yang rentan mengalami out-of-order saat jaringan mengalami latensi fluktuatif. Proyek ini menyelesaikan tantangan tersebut dengan pendekatan practical ordering (van Steen & Tanenbaum, 2023): event diurutkan berdasarkan timestamp penerimaan di server (ts_ingest) yang dipadukan dengan nomor ID berurut bawaan tabel database (auto-increment primary key). Skema ini memastikan urutan log yang stabil bagi pembaca tanpa memerlukan overhead protokol penanda waktu logis yang rumit seperti Lamport Timestamps.

4. 6 (Bab 6) – Model Kegagalan dan Strategi Crash Recovery

Failure modes dalam sistem ini mencakup kegagalan pengiriman (omission failure), kegagalan sistem (crash-stop), dan kegagalan jaringan (network partition) sebagaimana didefinisikan oleh Coulouris dkk. (2011). Strategi mitigasi yang diterapkan meliputi mekanisme durable state menggunakan named volume pada PostgreSQL. Jika container database atau API aggregator

mengalami crash-stop di tengah proses pengolahan, status pengolahan data tetap tersimpan di dalam penyimpanan fisik eksternal kontainer (van Steen & Tanenbaum, 2023). Begitu kontainer dibangun ulang (recreated), sistem langsung melanjutkan pemrosesan dari status terakhir tanpa adanya risiko pemrosesan ulang log yang telah selesai (avoiding duplicate reprocessing).

4.7 (Bab 7) – Konsistensi Eventual pada Agregasi Data

Sistem log aggregator ini menerapkan prinsip eventual consistency, di mana status statistik sistem dan ketersediaan data log pada endpoint /events tidak dijamin konsisten secara instan saat beban penulisan sedang berada pada puncaknya (van Steen & Tanenbaum, 2023). Ada jeda waktu (propagation delay) saat log disimpan ke status pending hingga selesai diproses oleh background workers ke status done (Coulouris dkk., 2011). Namun, karena didukung oleh mekanisme deduplikasi yang kuat, begitu beban penulisan dari publisher berhenti dan antrian pengolahan kosong, seluruh pembaca data dipastikan akan melihat data yang konsisten dan akurat secara menyeluruh (convergent state).

4.8 (Bab 8) – Integritas ACID dan Pencegahan Kegagalan Pembaruan Statistik

Untuk memelihara konsistensi internal basis data, setiap proses penerimaan batch log dari klien dijalankan di bawah naungan transaksi database ACID. Menurut Coulouris dkk. (2011), properti ACID (Atomicity, Consistency, Isolation, Durability) sangat krusial untuk menjamin integritas data dalam transaksi konkuren, di mana properti Atomicity menjamin bahwa seluruh operasi dalam suatu transaksi berhasil diselesaikan atau tidak sama sekali (all-or-nothing). Hal ini sejalan dengan konsep penanganan kegagalan transaksi terdistribusi yang dijelaskan oleh van Steen dan Tanenbaum (2023). Guna menghindari masalah kehilangan pembaruan (lost-update anomaly) pada counter statistik saat diakses secara konkuren—sebagaimana diklasifikasikan sebagai salah satu anomali transaksi utama oleh Coulouris dkk. (2011)—operasi pembaruan angka counter diinstruksikan secara atomik langsung di tingkat SQL (UPDATE stats SET val = val + delta WHERE ...). Pendekatan ini menghindari kesalahan pembacaan nilai lama (read-modify-write) yang kerap terjadi pada tingkat kode aplikasi.

4.9 (Bab 9) – Implementasi Pengendalian Konkurensi Tanpa Lock Aplikasi

Menggunakan mekanisme penguncian terdistribusi secara eksplisit di level kode aplikasi dapat memicu kompleksitas tinggi serta bahaya kebuntuan (deadlock). Sebagaimana dijelaskan oleh

Coulouris dkk. (2011) dalam bab pengendalian konkurensi, metode penguncian (locking) digunakan untuk mengelola akses simultan terhadap objek data bersama agar menjamin sifat serializability. Sebagai gantinya, pengendalian konkurensi dioptimalkan pada level mesin database relasional PostgreSQL melalui penerapan row-level locking dinamis dengan query `SELECT ... FOR UPDATE SKIP LOCKED` dikombinasikan dengan deklarasi unique constraint (van Steen & Tanenbaum, 2023). Pendekatan kontrol konkurensi ini mengombinasikan skema pencegahan konflik berbasis constraint unik (mirip dengan skema kontrol konkurensi optimistik) pada proses publish, dan pessimistic locking pada proses consume untuk menjamin pemrosesan antrian paralel yang aman, bebas dari balapan akses, dan memiliki overhead latensi yang sangat rendah tanpa memicu deadlock aplikasi (Coulouris dkk., 2011).

4. 10 (Bab 10–13) – Orkestrasi, Keamanan Jaringan Lokal, dan Observabilitas

Aspek orkestrasi didefinisikan menggunakan Docker Compose untuk mengelola daur hidup dan hubungan dependensi antarlayanan secara otomatis menggunakan health check berbasis perintah HTTP/SQL. Keamanan sistem dijamin melalui pembatasan akses jaringan lokal kontainer (`internal: true`), yang memutus jalur masuk langsung dari internet publik ke database PostgreSQL dan membatasi paparan hanya melalui gerbang aggregator port 8080 (van Steen & Tanenbaum, 2023). Observabilitas sistem didukung secara aktif melalui endpoint `/stats` untuk memantau metrik performa pengolahan serta penyediaan log server real-time terpusat untuk kebutuhan analisis anomali sistem terdistribusi (Coulouris dkk., 2011).

5. DAFTAR PUSTAKA

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). Distributed systems: Concepts and design (5th ed.). Pearson Education.

van Steen, M., & Tanenbaum, A. S. (2023). Distributed systems (4th ed.). Self-published.

6. Lampiran

Link yt : https://www.youtube.com/watch?v=4krL_np0mH0

Link github ; [github](#)